CANDIDATE NUMBER: 62094

Question 1: So I've chosen an Amazon style e-commerce platform based in the UK. I took inspiration from the Amazon sales report dataset in one of the provided Kaggle links.

We thus have 8 different entities: Customer, Address, Category, Product, Orders, Order_Item (a weak entity), Shipment and Payment.

- **Customer**: This is the entity which captures all of the basic information about clients. We have their first name, middle name initial and last name captured by a composite attribute called Customer_Name. This was done so to make it easier for someone using the database to sort by a specific part of the clients' names. This composite attribute hence splits into three other attributes: FName, MInitial, LName. Additionally, this entity includes other basic information about the customer such as their phone number, email address, registration date and whether or not they are a business; these are portrayed respectively by the attributes: Phone, Email, Registered_Date and Is_Business. Each customer has a customer ID which serves as a primary key for the entity, portrayed by the unique attribute: ID. Finally, phone number, email address and middle name initial are optional attributes as they aren't essential to this database.
- **Address**: This is the entity which stores the places where orders can be delivered. We have the city, country, postcode and state of each address, portrayed respectively by the attributes: City, Country, Postcode and State. Each address has an address ID which serves as a primary key, portrayed by the unique attribute: AddressID. It is separate from Customer because a customer can have several addresses thus avoiding repetition. Finally, postcode and state are optional attributes as not every address has them.
- **Category**: This is the entity which groups products into different types. We have the name of the category, portrayed by the attribute: Category_Name and a category ID which serves as a primary key, portrayed by the unique attribute: Category_Id.
- **Product**: This is the entity which captures the items for sale. We have the name, SKU, ASIN, current price and number of units in stock of each product, portrayed respectively by the attributes: Product_Name, SKU, ASIN, Unit_Price and Stock_Quantity. Each product has a product ID which serves as a primary key, portrayed by the unique attribute: Product_Id. The stock quantity lets the database check availability before an order is accepted. Finally, style and size are optional attributes as they do not apply to every product.
- **Orders**: This is the entity which captures a purchase made by a customer. We have the date it was placed and its status (pending, shipped, delivered, cancelled or returned), portrayed respectively by the attributes: Order_Date and Status. Each order has an order ID which serves as a primary key, which is portrayed by the unique attribute: Order_Id. The entity is called "Orders" as ORDER is a reserved word in SQL, so would unnecessarily complicate things later on.
- **Order_Item**: This is an entity which captures one product line of an order. It is however a weak entity as no line can exist without a pre-existing order and thus is attached to the Orders entity. We find the quantity, the amount paid for the line and the currency, portrayed respectively by the attributes: Quantity, Amount and Currency. The derived attribute Unit_Price_At_Purchase (amount divided by quantity) keeps the price the customer actually paid, even if the product's price changes later. As the entity is weak, it is identified by its order together with a line number, portrayed by the partially unique attribute: Line_Number.

- **Shipment**: This is the entity which captures the delivery of an order and only exists once the order has been sent out. We have the fulfilment method and the courier status, portrayed respectively by the attributes: Fulfilment and Courier_Status. Each shipment has a shipment ID which serves as a primary key, portrayed by the unique attribute: Shipment_Id. Finally, courier status is an optional attribute as a courier may not have reported a status yet.
- **Payment**: This is the entity which captures payments for orders. We have the method, amount, status and date of each payment, portrayed respectively by the attributes: Method, Amount, Payment_Status and Paid_At. Each payment has a payment ID which serves as a primary key, portrayed by the unique attribute: Payment_Id. It is separate from Orders so that an order can have several attempts. Finally, Paid_At is an optional attribute as declined payments have no payment date.

Relationships: A customer has zero or many addresses and makes zero or many orders, while each address and each order belongs to exactly one customer. Each order is delivered to exactly one address, while an address can receive zero or many orders. An order contains one or many order items through an identifying relationship and each order item is for exactly one product, which may appear in zero or many order items. An order has zero or many payments and zero or one shipment and each payment and shipment belongs to exactly one order. Finally, a category has one or many products and each product belongs to exactly one category. Optional attributes are stored as NULL when they are missing.

Question 2: i) When transitioning from the ER model to the relational schema we see first of all that the composite attribute which is Customer_Name disappears and just turns into its three subsidiary attributes (FName, MInitial, LName). Additionally, using the third part entity Order_Items solves the many to many relationship for ORDERS to PRODUCTS by turning it into two one to many relationships. We also see that the one to one relationship between SHIPMENT and ORDERS is solved by using order_id as a foreign key for SHIPMENT and labelling it as a unique value.

ii) For this e-commerce online database I decided to create 3 views.

- View 1: This is a view which shows the total amount which was spent on each order and how many lines that order comprised. This is important as the total amount inst stored anywhere else. It is built from ORDER_ITEM table and returns thus the order_id, number of lines (count) and order_total (sum of the amounts in those lines)
- View 2: This view shows the order history of a customer. This would be important for a customer who wants to see what they have bought and how much they've spent. It returns the customer's ID, first name and last name, followed by the order ID, order date, status and total of each of their orders(uses view 1).
- View 3: This view shows the remaining stock of each item and the number items it has sold. This is portrayed by the attributes: Product_Id, Product_Name, Category_Name, Stock_Quantity and Units_Sold. It is built from the PRODUCT and CATEGORY tables, joined through the category ID and from the ORDER_ITEM table, using a left join so that products which have never been sold still appear with 0 units sold.

iii) I made the email (from CUSTOMER) and SKU (from PRODUCT) attributes as unique as they can't be repeated but don't act as primary keys. Additionally, as I said beforehand, attributes like

email, MInitial, Postcode, State, Courier_Status, … are also labeled as optional in the relational schema, hence they are able to have no value at all (NULL). On the contrary, the primary and foreign keys are mandatory attributes which have to have a unique value inside, such as Address_Id, Order_Id, Shipment_Id, … I also have installed a unique setting to Order_Id in SHIPMENT which sets a maximum of one one shipment per order.

iv) As mentioned before, some values have been made optional and one has been made unique (email in CUSTOMER).

Question 3: After having generated the SQL code from the relational schema on ERDPlus, some minor changes were left to make that could only be made in the SQL code. On the one hand, I had to add a binary coding for whether or not a customer is a business or not (CHECK (Is_Business IN (0, 1))). Moreover, the same was done for the Payment_Status where the value had to be either “Accepted” or “Declined”. On the other hand, I had to make sure that any payment, amount and price value was a positive value, which meant adding for example: CHECK (Unit_Price >= 0).

Once that was done I first of all copy pasted the completed SQL code into DB Browser and ran the whole code. It was only after the tables were created that I one by one implemented the views starting from View 1 to the third one.

Question 4: All of the data in this database has been generated. The Amazon sales dataset has no order IDs, customers, addresses or payments so most of the tables would’ve needed generated data anyway and mixing two sources risked inconsistencies. This thus gave me full control over the edge cases as I could generate everything from top to bottom to make the dataset realistic and give meaningful results. I used a Python script (generate_data.py) with the random module and the Faker library (https://www.datacamp.com/tutorial/creating-synthetic-data-with-python-faker-tutorial). The seed is fixed.

How every table was created:

- **CATEGORY:** I created a dictionary of different categories. Each category had a list of items, a minimum and maximum price that could be demanded for each product. I then created a for loop which gave each category an ID number.
- **PRODUCT:** I created multiple lists which represented the different adjectives, colours and sizes. I then ran a for loop which created 30 products per category. The names of these products were made by combining a random name with a product type fitting that category. The size, colour and price were all random sensible values within the specific guidelines I had set for the categories (eg. price between min and max value). Furthermore, the SKU uses the first three letters of the category and the product number and the ASIN follows Amazon’s format (“B0” plus 8 digits). Only clothes get a size (NULL otherwise). Stock is 20 to 500 units, except 5% of products with 0 to 5 to give the restock trigger something to detect and also mimic reality. Each product also gets a popularity weight (10% bestsellers, 30% fairly popular, the rest normal) used when creating orders.
- **CUSTOMER:** For the customer data that's where I used the Faker python library. Using this, I created a for loop ranging from 1 to number of customers+1 which would create fake names, email and phone numbers. A customer would be labeled as a business if, given a random proportion between 6 and 12% of customers, their name was associated

with a “1”. Now to make the data more realistic, I also set a 40% probability that the person filled in a random middle name initial. Furthermore, using a random roll I gave each customer an ordering weight (10% order a lot, 30% fairly often, the rest rarely) and flagged 15% as “quiet”, meaning they stop ordering after June 2025, which gives the win-back query, which will be explained later on, something to find. Also, registration dates are between 2023 and 2024.

- **ADDRESS:** Once again using the Faker python library, in a for loop, i generated fake cities, postcodes and counties. These are all situated in the United Kingdom as this is a UK online e-commerce platform. Each customer gets 1 to 3 addresses as some have more than one house.
- **ORDERS, ORDER_ITEM and SHIPMENT:** These 3 were created together in one loop of 3000 orders. Each order receives a sequential ID (ORD-000001+) and a random weighted status. It is tied to a specific customer based on ordering weights using only their own address, with a random 2025 date. There is exactly one item which is chosen via popularity weights in every order, alongside a quantity of 1-3 and a total amount in pounds. Only shipped, delivered and returned orders generate a shipment, while pending and cancelled orders receive none.
- **PAYMENT:** I started by creating a list of all the different payment methods. Then using a for loop over every order, I pick a random method and set the payment from the order status (cancelled order -> declined payment, any other -> accepted payment) plus a 5% chance that a declined attempt came first to make it more realistic. And that’s why some orders have two payments. Declined payments have no Paid_At date.

So finally, I turned the lists into pandas DataFrames and loaded them into the empty database (parent tables first: CUSTOMER, CATEGORY). For consistency purposes, I also ran a PRAGMA foreign_key_check which returned no violations.

Question 5:

Q5 Use case 1: A customer checking their own account

When I’m checking my account I like to know how much I've spent so far to see if I’m still within my budget and whether I should be buying something else. So this is why having a query which shows the running total of what the customer has spent on his account is so important. The way I did it was to make a windows function which adds up the amounts in date order while still numbering the orders (as in 1st order, 2nd order,...). I also made sure the cancelled and returned orders weren't taken into account. Additionally, I also sorted by Order_Id so that 2 orders on the same day still get their own running total.

Equally as important when I’m checking my account, if for example I’m realising that I’m spending too much, having a way to see what are the categories I am buying the most to maybe try to restrict myself when looking at these items. That is why the second query shows exactly that. It follows the chain ORDERS, ORDER_ITEM, PRODUCT and CATEGORY with joins, groups the rows by category name and for each one counts the orders, adds up the items bought and the money spent. It is then finally sorted by total spent. Once again, canceled and refunded orders aren’t taken into account.

Query 1:

	Order_Number	Order_Id	Order_Date	Amount	Running_Total
1	1	ORD-002132	2025-03-08	21.62	21.62
2	2	ORD-002634	2025-03-17	28.25	49.87
3	3	ORD-002822	2025-07-08	28.47	78.34
4	4	ORD-001788	2025-10-24	49.23	127.57
5	5	ORD-002757	2025-12-22	55.32	182.89

Query 2:

	Category_Name	Number_Of_Orders	Items_Bought	Total_Spent
1	Toys	2	3	76.94
2	Books	2	4	56.72
3	Clothing	1	1	49.23

Q5 Use case 2: A member of staff managing orders and stock

If I was an employee, one of my big concerns would be the order pipeline. As in making sure that there is a normal proportion of parcels delivered, shipped, pending… This is why the first query makes that task a lot easier. It allows them to see how many orders are in each status and what share of all orders it represents. It uses GROUP BY Status with COUNT(*) and a sum window function over the grouped counts, to get the total number of orders so each count can be turned into a percentage.

Now equally as important for a member of staff is to make sure every product is being constantly restocked. As if one isn't restocked in time and during that absence a customer wanted that product, you lose that client. This query lists the products with 5 or fewer units left, with their category and how many units they have sold, so staff know what to restock first. A left join to ORDER_ITEM keeps products that have never been sold and COALESCE turns their empty total into 0. The result is sorted by units sold so that the best sellers at the top are the most urgent.

Query 1:

	Status	Number_Of_Orders	Percentage_Of_Orders
1	Delivered	1790	59.7
2	Shipped	437	14.6
3	Cancelled	316	10.5
4	Pending	286	9.5
5	Returned	171	5.7

Query 2:

	Product_Id	Product_Name	Category_Name	Stock_Quantity	Units_Sold
1	146	Smart Toy Car	Toys	4	45
2	71	Deluxe Blender	Home and Kitchen	2	42
3	157	Compact Perfume	Beauty	0	27
4	52	Compact Jumper	Clothing	2	25
5	141	Classic Puzzle	Toys	2	19
6	11	Premium Webcam	Electronics	2	10
7	174	Classic Perfume	Beauty	5	9
8	134	Essential Doll	Toys	3	7

Q5 Use case 3: Sales overview and marketing

Knowing what your most profitable category is essential for someone on the Sales and Marketing team. If you don't, you not only run the risk of missing out on profits for the booming sectors but also miss out on the opportunity to see which category isn't performing so well and finding a solution to that. The first query does that. It uses the same join chain as 1.2 but for all customers, grouped by category and shows: the number of orders, the total revenue and the average order value. So it can show whether a category is doing good because of volume or because of value.

Now specifically on the marketing team, one issue you could encounter is having a lot of customers registered to the e-commerce platform, but having only a small proportion of those clients placing orders. The second query addresses that as it finds customers who have never placed an order so that the marketing can find a solution (e.g. a discount). A subquery lists every customer ID that appears in ORDERS and the outer query keeps the customers who are not in that list. Sorting by registration date and limiting the output to 10, puts the top 10 customers who have waited longest first.

Query 1:

	Category_Name	Number_Of_Orders	Revenue	Average_Order_Value
1	Electronics	611	139564.33	228.42
2	Home and Kitchen	365	42628.17	116.79
3	Clothing	426	33318.58	78.21
4	Beauty	285	26064.29	91.45
5	Toys	378	21222.5	56.14
6	Books	448	14558.17	32.5

Query 2:

	ID	FName	LName	Email	Registered_Date
1	397	Gail	Davidson	lynchbryan@example.com	2023-01-07
2	291	Claire	Rose	qclark@example.com	2023-01-11
3	202	Lewis	Bentley	staylor@example.org	2023-01-17
4	470	Leah	Jenkins	bradley83@example.com	2023-01-27
5	303	Jodie	Powell	geraldine44@example.net	2023-01-30
6	319	Deborah	Harris	leonardshane@example.com	2023-02-01
7	462	Ashleigh	Connor	mark88@example.org	2023-03-18
8	328	Melissa	Johnson	xmurphy@example.org	2023-06-19
9	140	Robin	Patel	xcook@example.com	2023-06-24
10	396	Abbie	Ellis	roy28@example.org	2023-07-20

Question 6:

Q6 Triggers 1 and 2: Stock availability

As a customer, if I order a product just for it to turn out to be out of stock afterwards, it would be frustrating. That's what happens if the database doesn't stop you from ordering more than what it is actually in stock. This goes hand in hand with another issue, which is if the database doesn't update the stock once a product has been purchased.

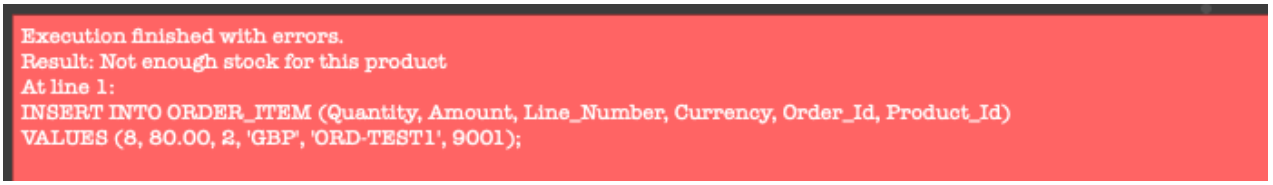
That's why I used 2 triggers on ORDER_ITEM. The first one runs before a line is inserted and compares the quantity ordered with the stock of that product. If the quantity is bigger, it stops the insert and raises the error "Not enough stock for this product". The second one runs after a valid line is inserted and takes the quantity out of the stock.

Passing test: ordering 3 units of a product with 10 in stock goes through, the stock becomes 7



	Product_Id	Stock_Quantity
1	9001	7

Failing test: ordering 8 units when only 7 are left raises the error and the stock stays at 7



```
Execution finished with errors.
Result: Not enough stock for this product
At line 1:
INSERT INTO ORDER_ITEM (Quantity, Amount, Line_Number, Currency, Order_Id, Product_Id)
VALUES (8, 80.00, 2, 'GBP', 'ORD-TEST1', 9001);
```

Q6 Triggers 3 and 4: The delivery address must belong to the customer

Equally as important, if I place an order and it ends up at a stranger's house, that would also be frustrating. In this database an order stores both the customer who placed it and the address it is delivered to. The foreign keys on their own only check that each of them exists and don't check whether or not they belong together.

That's why I made 2 triggers on ORDERS. The first one runs before a new order is inserted and it looks up which customer owns the chosen address and compares it to the customer placing the order. If they are different, it stops the insert with the error "The delivery address does not belong to this customer". The second one does exactly the same check before an existing order is updated as someone could place a valid order and then just simply change the address afterwards to somebody else's.

Passing test: customer 1 places an order using one of their own addresses and the order is created

	Order_Id	Customer_Id	AddressID
1	ORD-TEST1	1	1
2	ORD-TEST2	1	1

Failing test: customer 1 places an order using customer 2's address, which raises the error and no order is created

```
Execution finished with errors.
Result: The delivery address does not belong to this customer
At line 1:
INSERT INTO ORDERS (Order_Id, Order_Date, Status, ID, AddressID)
VALUES ('ORD-TEST3', '2026-12-31', 'Pending', 1,
    (SELECT MIN(AddressID) FROM ADDRESS WHERE ID = 2));
```

Statement on the use of generative AI

I used Claude (Anthropic) as an assistant during this assignment. It was used to: discuss whether to use real or synthetic data and how to structure the generation process; debug errors in my data generation script (a nested loop and an incorrect variable in the customer generation code); explain the assignment brief and the marking criteria; suggest consistency checks for the database; suggest a structure for this report; and add short explanatory comments to my SQL and Python files.

The database design, [the SQL queries and triggers,] and the data generation logic are my own work, and I ran and validated all code and results myself. The full chat logs are included in the submission

Google AI Mode: to generate the input lists used by my data generation script: the product catalogue (catalogue: category names and product types for each category), and the lists of product adjectives, colours and clothing sizes (adjectives, colours, sizes).

REFERENCES:

https://www.datacamp.com/tutorial/creating-synthetic-data-with-python-faker-tutorial

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_sql.html

https://www.w3schools.com/python/module_random.asp

https://www.w3schools.com/sql/sql_syntax.asp

https://www.w3schools.com/sql/sql_view.asp

https://www.geeksforgeeks.org/sql/sql-trigger-student-database/

https://erdplus.com

https://sqlitebrowser.org